

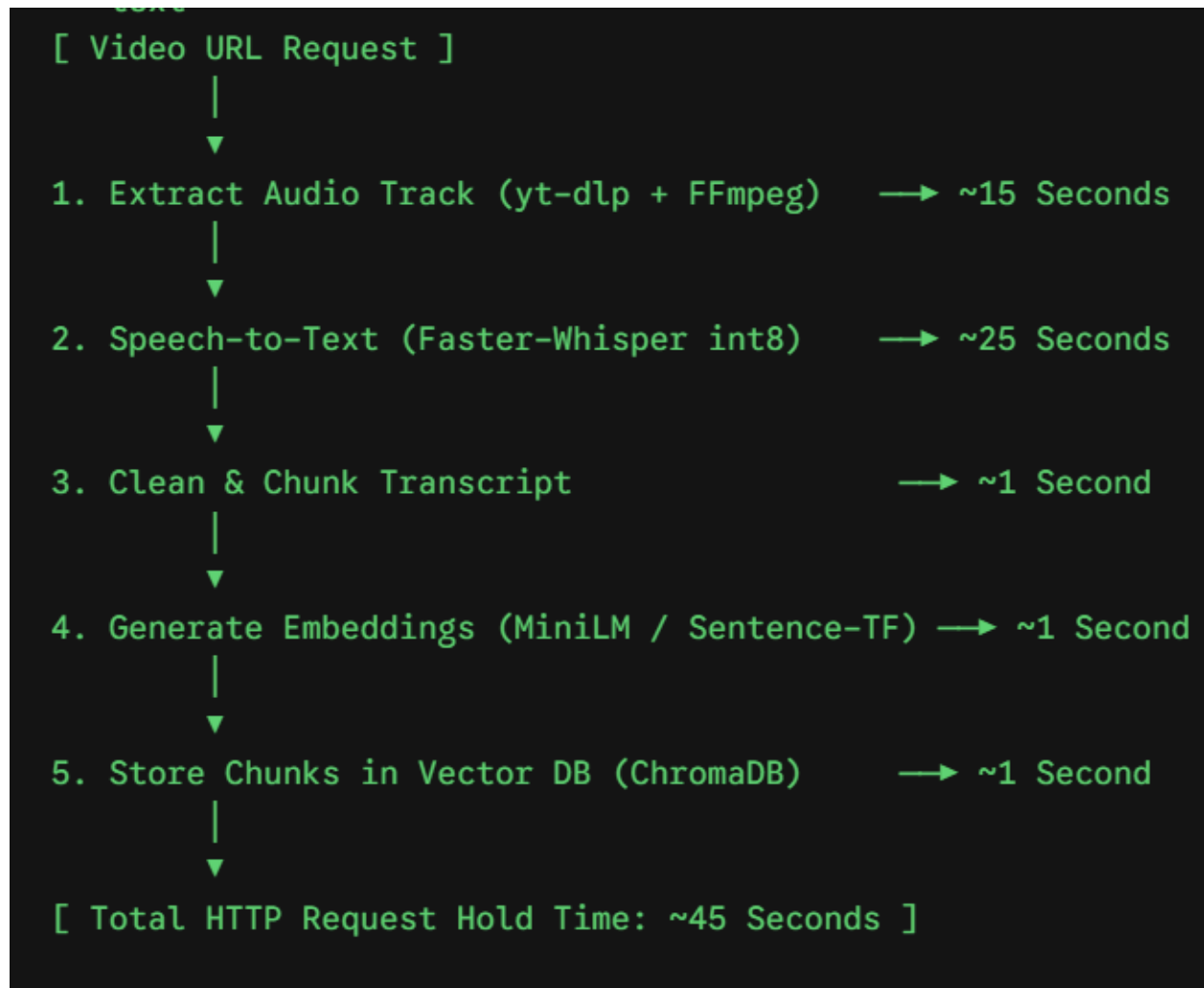
RAG Ingestion Architecture

Background Job Pipelines & Chunking Optimization Rules

Part 1: Asynchronous Video & Document Ingestion via Background Jobs

1. The Core Architecture Problem :

In a standard synchronous FastAPI endpoint, long-running operational pipelines cause severe system bottlenecks. For instance, ingesting a single 5-minute educational video requires executing sequential steps:



Primary Risks of Synchronous Ingestion:

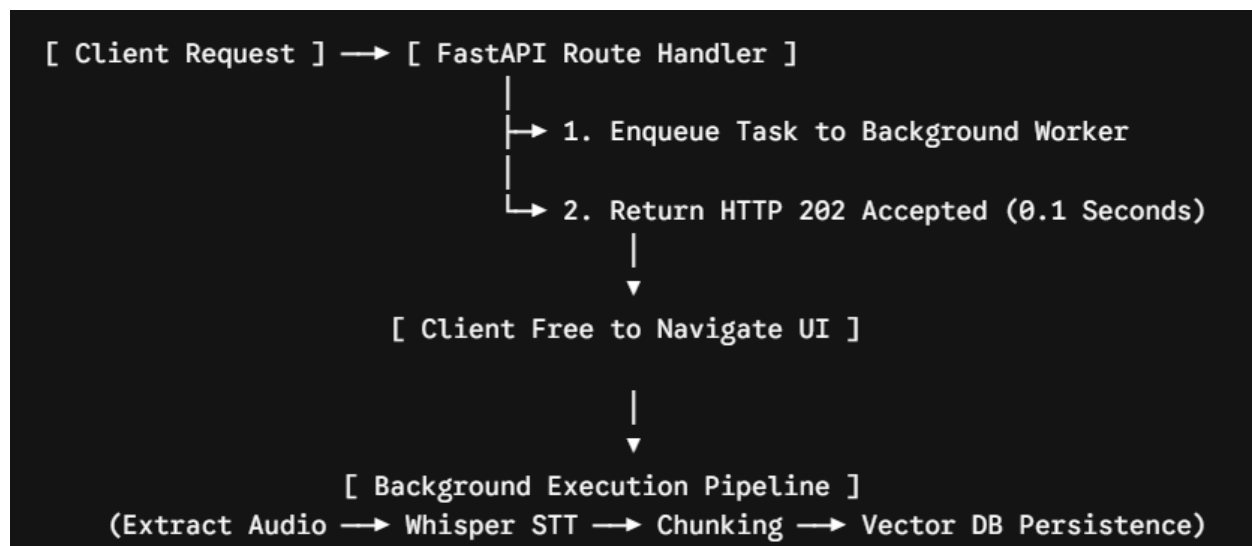
- **HTTP Gateway Timeouts:** Proxies, load balancers (e.g., Nginx, AWS ALB), and browser HTTP clients default to socket timeout thresholds between 30 and 60 seconds. A 10-minute video or a 100-page PDF will breach this threshold, throwing a 504 Gateway

Timeout even if the processing continues on the server.

- **UI Thread Locking:** The user interface remains stuck on a loading spinner. The user cannot navigate the dashboard, review existing collections, or interact with the RAG chat system until the request completes.
- **Event Loop Blocking:** Executing heavy, blocking CPU/IO tasks inside the main FastAPI async worker thread halts execution for concurrent incoming traffic.

2. The Solution: Non-Blocking Background Processing

Background job execution decouples the HTTP Request/Response lifecycle from data ingestion. Heavy sequential steps remain sequential, but they run silently on worker threads while the client gets an immediate acknowledgment.



Part 2: RAG Chunking Strategy & Overlap Optimization Rules

1. Conceptual Framework & Real-World Analogy

When ingesting documents into a Retrieval-Augmented Generation (RAG) system, raw files (PDFs, Markdown, Video Transcripts) cannot be processed as a single entity due to context window boundaries and vector similarity precision requirements.

The Jaggery (Gud) Block Analogy:

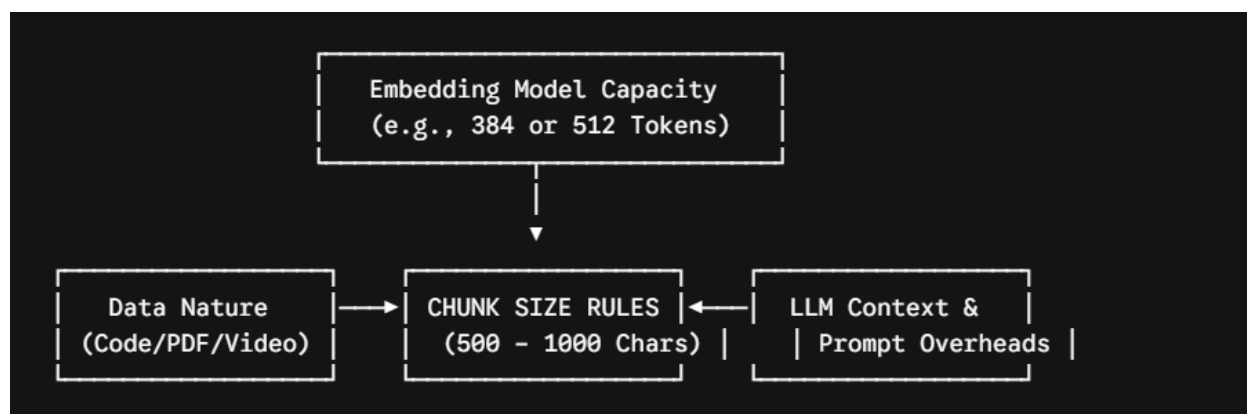
Imagine you possess a **10 Kilogram Block of Jaggery**.

- **The Storage Constraint:** You cannot fit a solid 10 kg block into a standard refrigerator compartment. It must be chopped into uniform **500-gram blocks** (This is your **Chunk Size**).

- **Contextual Continuity:** If two distinct blocks break cleanly along a boundary, you lose the structural continuity between adjacent layers. To retain flavor transition, each block shares a **50-gram overlapping layer** with its neighbor (This is your **Chunk Overlap**).
- **Granular Access:** When someone requests sweetening, you do not pull out the entire 10 kg block. You retrieve only the specific 1-2 small blocks that match the required profile (**Top-K Vector Retrieval**).

2. Strategic Determinants of Chunk Size & Overlap

The selection of Chunk Size and Overlap is **never an arbitrary guess**. It is strictly governed by three technical constraints:



A. Embedding Model Token Limit (Hard Upper Bound)

- Models like all-MiniLM-L6-v2 or bge-small-en have a maximum sequence length limit of **256 to 512 tokens**.
- If you set a chunk size of 2,000 characters (~500 tokens), the embedding model will **silently truncate** all text past its maximum token limit, causing irreversible context loss during vector generation.

B. LLM Context Window Budgeting

- When answering a user prompt, the RAG engine injects the retrieved chunks into the prompt system message.
- If Top-K = 10 and Chunk Size = 1500, you inject 15,000 characters (~3,750 tokens) of context alone. Balancing this against user queries, chat history, and system instructions is critical to preventing context window exhaustion or "lost in the middle" phenomena.

C. Nature of Source Content

- **Technical Books / Legal PDFs:** Highly dense information. Requires smaller chunk sizes (300–500 characters) with high overlap (10–15%) to maintain sentence-level precision.
- **Conversational Transcripts / Videos:** Sparse information density with long padding phrases. Works best with moderate chunk sizes (500–1000 characters) and medium overlap (50–100 characters).

3. Scale Management & Vector Database Safeguards

Multi-Page Document Ingestion (e.g., 1,400-Page PDFs)

- As document size scales to thousands of pages, **the Chunk Size (e.g., 500) and Overlap (e.g., 50) remain constant.**
- Scale only increases the total volume of generated chunks (e.g., expanding from 7 chunks to 15,000 chunks).

Vector DB Ingestion Safeguard: Batch Processing

Pushing tens of thousands of vectors into a local SQLite-backed Vector DB (such as ChromaDB) in a single synchronous call triggers thread deadlocks, memory spikes, or database lock errors.

The Golden Ingestion Pattern (Batching):

```
def store_in_chromadb_batched(chunks: list, embeddings: list, batch_size: int = 200) -> None:
```

```
    """
```

Ingests generated document chunks in safe, memory-efficient batches to avoid database thread-locking and memory spikes.

```
    """
```

```
    total_chunks = len(chunks)
```

```
    logger.info(f"Starting batched insertion for {total_chunks} chunks.")
```

```
    for i in range(0, total_chunks, batch_size):
```

```
        batch_chunks = chunks[i : i + batch_size]
```

```
        batch_embeddings = embeddings[i : i + batch_size]
```

```
        batch_texts = [c['text'] for c in batch_chunks]
```

```
        batch_metadatas = [c['metadata'] for c in batch_chunks]
```

```
        batch_ids = [str(uuid.uuid4()) for _ in batch_chunks]
```

```

db.add_documents(
    collection_name=Collections.VIDEO_CHUNKS,
    documents=batch_texts,
    embeddings=batch_embeddings,
    metadatas=batch_metadatas,
    ids=batch_ids
)
logger.debug(f"Ingested batch {i // batch_size + 1} / {(total_chunks + batch_size - 1) // batch_size}")

```

4. Summary Architectural Standards

Parameter	Recommended Standard	Engineering Rationale
Default Chunk Size	500 - 1000 Characters	Fits safely within standard embedding model limits (e.g., MiniLM 256-512 tokens) without text truncation.
Default Chunk Overlap	10% - 15% of Chunk Size	Prevents semantic boundary cuts between adjacent sentence fragments.
Ingestion Execution	Asynchronous (BackgroundTasks)	Guarantees instant sub-second API response time and eliminates HTTP 504 Gateway Timeouts.

Vector DB Pipeline	Batched Write Operations (batch_size=200)	Protects memory footprint and eliminates SQLite database locking under heavy loads.
---------------------------	--	--